Node.js and Express Basics

Build a small REST API

A concise, original learning guide with practical examples and a mini project.

Designed for beginners • 3 learning pages + cover

1. Node.js Foundations

Node.js runs JavaScript outside the browser. It is widely used for APIs, command-line tools, automation, and server-side applications.

Create a project

```
mkdir library-api\ncd library-api\nnpm init -y\nnpm install express
```

npm manages project dependencies. Keep package.json and the lockfile in source control so installs remain reproducible.

A basic server

```
const express = require("express");\nconst app = express();\nconst PORT = process.env.PORT || 3000;\n\n
app.get("/", (req, res) => {\n  res.send("Library API is running");\n});\n\napp.listen(PORT, () => cons
ole.log(`Listening on ${PORT}`));
```

A server listens for HTTP requests and sends responses. Use environment variables for deployment-specific settings such as the port.

2. Routes and JSON

Create a JSON endpoint

```
app.use(express.json());\n\nconst books = [\n  { id: 1, title: "Python Basics", author: "A. Shah" },\n
 { id: 2, title: "React Guide", author: "R. Patil" }\n];\n\napp.get("/api/books", (req, res) => {\n  re
s.json(books);\n});
```

express.json() parses JSON request bodies. res.json() sends JSON to the client. In a real application, data should usually come from a database or persistent storage.

Search with query parameters

```
app.get("/api/books/search", (req, res) => {\n  const q = String(req.query.q || "").toLowerCase();\n  c
onst result = books.filter(book =>\n    book.title.toLowerCase().includes(q) ||\n    book.author.toLowe
rCase().includes(q)\n  );\n  res.json(result);\n});
```

Query parameters are useful for filtering and pagination. Validate them and limit results when working with large datasets.

HTTP status codes

Use 200 for successful reads, 201 for created resources, 400 for invalid input, 404 when a resource is missing, and 500 for unexpected server errors.

3. Safety and Reliability

Validate incoming data

Never trust request data. Check required fields, data types, lengths, and allowed values before storing information. Use a validation library as the API grows.

Error handling

```
app.get("/api/books/:id", (req, res) => {\n  const id = Number(req.params.id);\n  const book = books.fi
nd(item => item.id === id);\n  if (!book) return res.status(404).json({ error: "Book not found" });\n
res.json(book);\n});
```

Return useful status codes and safe error messages. Avoid exposing stack traces or private server details to users.

Production checklist

Use HTTPS in deployment, configure CORS intentionally, keep dependencies updated, add request logging, use a persistent database, and protect sensitive endpoints with authentication and authorization.

Mini project

Build CRUD endpoints for books: GET all, GET by ID, POST a new book, PUT/PATCH an update, and DELETE a book. Test each endpoint with an API client.

This guide uses an in-memory array for learning. Data stored only in memory is lost when the server restarts.